



ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



Discrete Mathematics

PART 1

COMBINATORIAL THEORY

(Lý thuyết tổ hợp)

PART 2

GRAPH THEORY

(Lý thuyết đồ thị)

Contents of Part 1

Chapter 0: Sets, Relations

Chapter 1: Counting problem

Chapter 2: Existence problem

Chapter 3: Enumeration problem

Chapter 4: Combinatorial optimization problem

CONTENTS

- 1. Introduction to problem**
2. Algorithm and Complexity
3. Generating algorithm
4. Backtracking algorithm

Introduction to problem

- A problem asking to give a list of all combinations that satisfy a given number of properties is called the **combinatorial enumeration problem**.
- As the number of combinations to enumerate is usually very large even when the configuration size is not large:
 - The number of permutations of n elements is $n!$
 - The number of subset consisting m elements taken from n elements is $n!/(m!(n-m)!)$

Therefore, it is necessary to have the concept of solving combinatorial-enumeration problems

Introduction to problem

- The combinatorial enumeration problem is solvable if we can define *an algorithm* so that all configurations could be built in turn.
- An enumeration algorithm must satisfy 2 basic requirements:
 - Do not repeat a configuration,
 - Do not miss a configuration.

CONTENTS

1. Introduction to problem
- 2. Algorithm and Complexity**
3. Generating algorithm
4. Backtracking algorithm

Algorithm

- All algorithms must satisfy the following criteria:
 - (1) **Input**. The algorithm receives data from a certain set.
 - (2) **Output**. For each set of input data, the algorithm gives the solution to the problem.
 - (3) **Precision**. Each instruction is clear and unambiguous.
 - (4) **Finiteness**. If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite (possibly very large) number of steps.
 - (5) **Uniqueness**. The intermediate results of each step of the algorithm are uniquely determined and depend only on the input and the result of the previous steps.
 - (6) **Generality**. The algorithm could be applied to solve any problem with a given form

Comparing Algorithms

- Given 2 or more algorithms to solve the same problem, how do we select the best one?
- Some criteria for selecting an algorithm:
 - 1) Is it easy to implement, understand, modify?
 - 2) How long does it take to run it to completion? **TIME**
 - 3) How much of computer memory does it use? **SPACE**

In this lecture we are interested in the second and third criteria:

- **Time complexity:** The amount of time that an algorithm needs to run to completion
- **Space complexity:** The amount of memory an algorithm needs to run

We will occasionally look at space complexity, but we are mostly interested in time complexity in this course. Thus in this course the better algorithm is the one which runs faster (has smaller time complexity)

How to Calculate Running time

- Most algorithms transform input objects into output objects



The running time of an algorithm typically grows with the input size

- Idea: analyze running time as a function of input size
- Even on inputs of the same size, running time can be very different

Example: algorithm that finds the first prime number in an array by scanning it left to right

- Array 1: 3 9 8 12 15 20 (the algorithm stops once the 1st element is considered)
- Array 2: 9 8 3 12 15 20 (the algorithm stops once the 3rd element is considered)
- Array 3: 9 8 12 15 20 3 (the algorithm stops until the last element is considered)

➔ Idea: analyze running time in the

- best case
- worst case
- average case

Kind of analyses

Best-case:

- $T(n)$ = minimum time required to execute the algorithm on any input of size n
- Treat with a slow algorithm that works fast on some input.

Average-case: (sometimes)

- $T(n)$ = expected average time of algorithm over all inputs of size n .
- Need assumption of statistical distribution of inputs
- Very useful but often difficult to determine

Worst-case: (usually)

- $T(n)$ = maximum time of algorithm on any input of size n .
- Easier to analyze

To evaluate the running time: 2 ways:

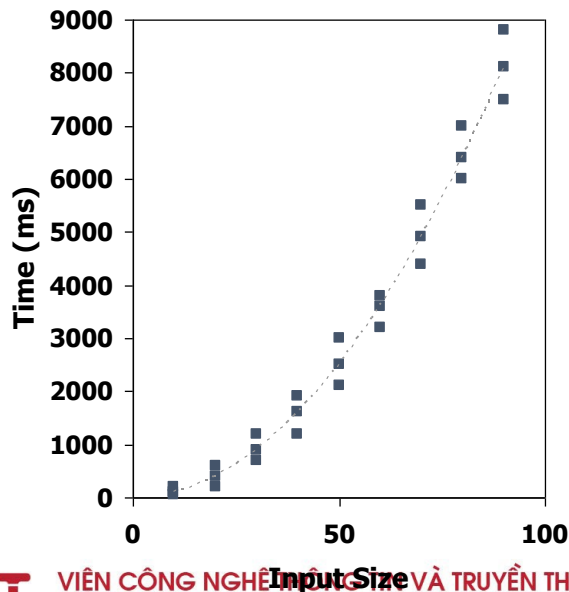
- Experimental evaluation of running time
- Theoretical analysis of running time

Experimental Evaluation of Running Time

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Use a method like `clock()` to get an accurate measure of the actual running time

```
clock_t startTime = clock();  
doSomeOperation();  
clock_t endTime = clock();  
clock_t clockTicksTaken = endTime - startTime;  
double timeInSeconds = clockTicksTaken / (double) CLOCKS_PER_SEC;
```

- Plot the results



Limitations of Experiments when evaluating the running time of an algorithm

- Experimental evaluation of running time is very useful but
 - It is necessary to implement the algorithm, which may be difficult
 - Results may not be indicative of the running time on other inputs not included in the experiment
 - In order to compare two algorithms, the same hardware and software environments must be used
- ➔ We need: **Theoretical Analysis of Running Time**

Theoretical Analysis of Running Time

- Uses a pseudo-code description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment (Changing the hardware/software environment affects the running time by a constant factor, but does not alter the growth rate of the running time)

Asymptotic notation

$\Theta, \Omega, O, o, \omega$

- What these symbols do are:
 - give us a notation for talking about how fast a function goes to infinity, which is just what we want to know when we study the running times of algorithms.
 - defined for functions over the natural numbers
 - used to compare the order of growth of 2 functions
- Example:** $f(n) = \Theta(n^2)$: Describes how $f(n)$ grows in comparison to n^2 .
- Instead of working out a complicated formula for the exact running time, we can just say that the running time is for example $\Theta(n^2)$ [read as theta of n^2]: that is, the running time is proportional to n^2 plus lower order terms. For most purposes, that's just what we want to know.

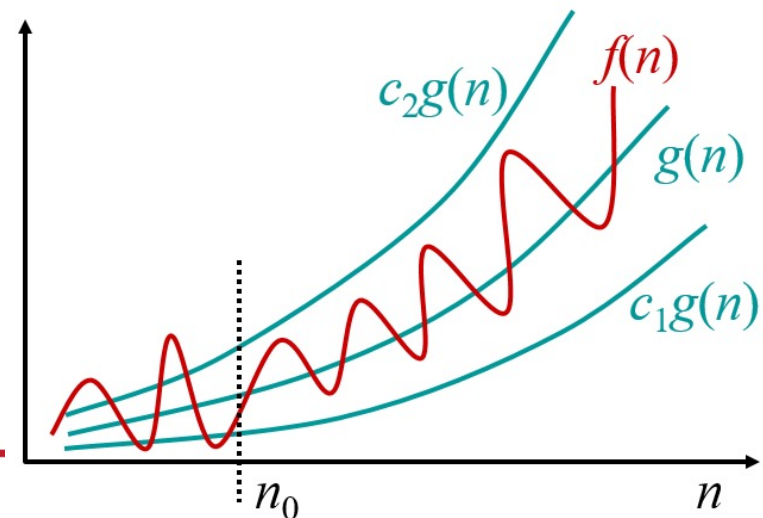
Θ - Theta notation

- For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions

$$\Theta(g(n)) = \left\{ f(n) : \begin{array}{l} \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ s.t.} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

Intuitively: Set of all functions that have the same *rate of growth* as $g(n)$.

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be “sand-wiched” between $c_1 g(n)$ and $c_2 g(n)$ for sufficiently large n
 - $f(n) = \Theta(g(n))$ means that there exists some constant c_1 and c_2 s.t.
 $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for large enough n .
- When we say that one function is theta of another, we mean that neither function goes to infinity faster than the other.



$$f(n) = \Theta(g(n)) \iff \exists c_1, c_2, n_0 > 0 : \forall n \geq n_0, c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Example 1: Show that $10n^2 - 3n = \Theta(n^2)$

- With which values of the constants n_0, c_1, c_2 then the inequality in the definition of the theta notation is correct:

$$c_1 n^2 \leq f(n) = 10n^2 - 3n \leq c_2 n^2 \quad \forall n \geq n_0$$

- Suggestion: Make c_1 a little smaller than the leading (the highest) coefficient, and c_2 a little bigger.

→ Select: $c_1 = 1, c_2 = 11, n_0 = 1$ then we have

$$n^2 \leq 10n^2 - 3n \leq 11n^2, \text{ with } n \geq 1.$$

→ $\forall n \geq 1: 10n^2 - 3n = \Theta(n^2)$

- Note:** For polynomial functions: To compare the growth rate, it is necessary to look at the term with the highest coefficient

$$f(n) = \Theta(g(n)) \iff \exists c_1, c_2, n_0 > 0 : \forall n \geq n_0, c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Example 2: Show that $f(n) = \frac{1}{2}n^2 - 3n = \Theta(n^2)$

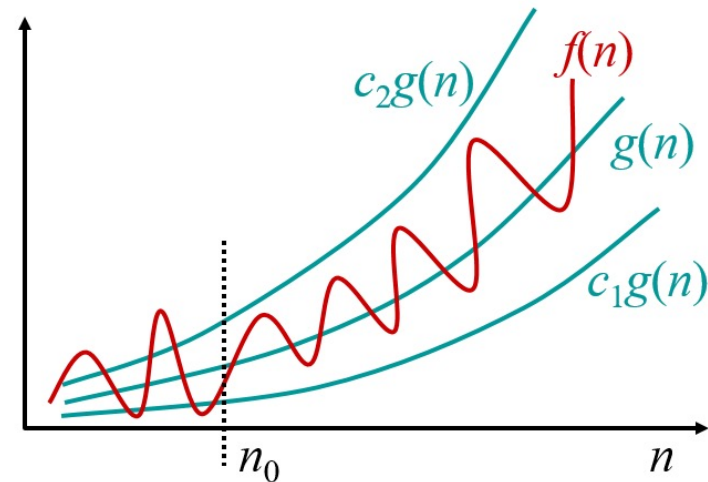
We must find n_0 , c_1 and c_2 such that

$$c_1 n^2 \leq f(n) = \frac{1}{2}n^2 - 3n \leq c_2 n^2 \quad \forall n \geq n_0$$

Select $c_1 = 1/4$, $c_2 = 1$, and $n_0 = 7$ we have:

$$\frac{1}{4}n^2 \leq f(n) = \frac{1}{2}n^2 - 3n \leq n^2 \quad \forall n \geq 7$$

$$\Rightarrow \forall n \geq 7: \frac{1}{2}n^2 - 3n = \Theta(n^2)$$



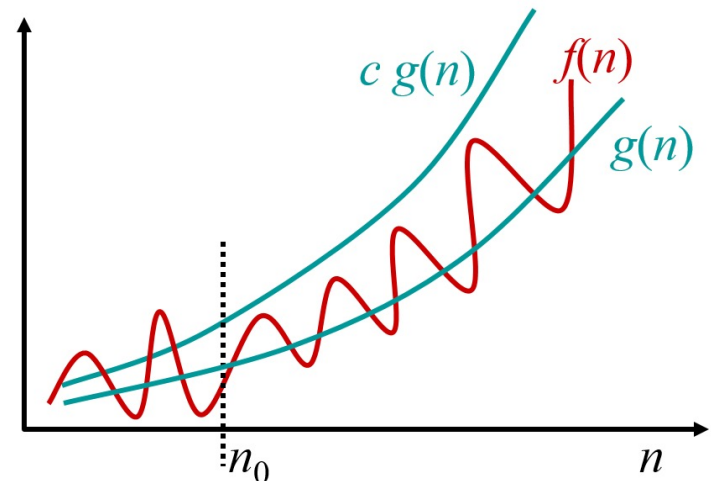
O - big Oh notation

For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$$O(g(n)) = \left\{ f(n) : \begin{array}{l} \text{there exist positive constants } c \text{ and } n_0 \text{ s.t.} \\ 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

Intuitively: Set of all functions whose *rate of growth* is the same as or lower than that of $g(n)$.

- We say: $g(n)$ is asymptotic upper bound of the function $f(n)$, to within a constant factor, and write $f(n) = O(g(n))$.
- $f(n) = O(g(n))$ means that there exists some constant c such that $f(n)$ is always $\leq cg(n)$ for large enough n .
- $O(g(n))$ is the set of functions that go to infinity no faster than $g(n)$.



Big-Oh Examples

$O(g(n)) = \{f(n) : \exists \text{ positive constants } c \text{ and } n_0, \text{ such that } \forall n \geq n_0, \text{ we have } 0 \leq f(n) \leq cg(n) \}$

- Example 1: Show that $2n + 10 = O(n)$

→ $f(n) = 2n + 10, g(n) = n$

- Need constants c and n_0 such that $2n + 10 \leq cn$ for $n \geq n_0$
- $(c - 2)n \geq 10$
- $n \geq 10/(c - 2)$
- Pick $c = 3$ and $n_0 = 10$

Note

- The values of positive constants n_0 and c **are not unique** when proof the asymptotic formulas
- Example: show that $100n + 5 = O(n^2)$
 - $100n + 5 \leq 100n + n = 101n \leq 101n^2 \quad \forall n \geq 5$
 $n_0 = 5$ and $c = 101$ are constants need to determine
 - $100n + 5 \leq 100n + 5n = 105n \leq 105n^2 \quad \forall n \geq 1$
 $n_0 = 1$ and $c = 105$ are also constants need to determine
- Only need to find **some** positive constants c and n_0 satisfying the equality in the definition of asymptotic notation

Big-Oh Examples

$O(g(n)) = \{f(n) : \exists \text{ positive constants } c \text{ and } n_0, \text{ such that } \forall n \geq n_0, \text{ we have } 0 \leq f(n) \leq cg(n) \}$

- Example 2: Show that $3n^3 + 20n^2 + 5$ is $O(n^3)$

Need constants c and n_0 such that $3n^3 + 20n^2 + 5 \leq cn^3$ for $n \geq n_0$
this is true for $c = 4$ and $n_0 = 21$

- Example 3: Show that $3 \log n + 5$ is $O(\log n)$

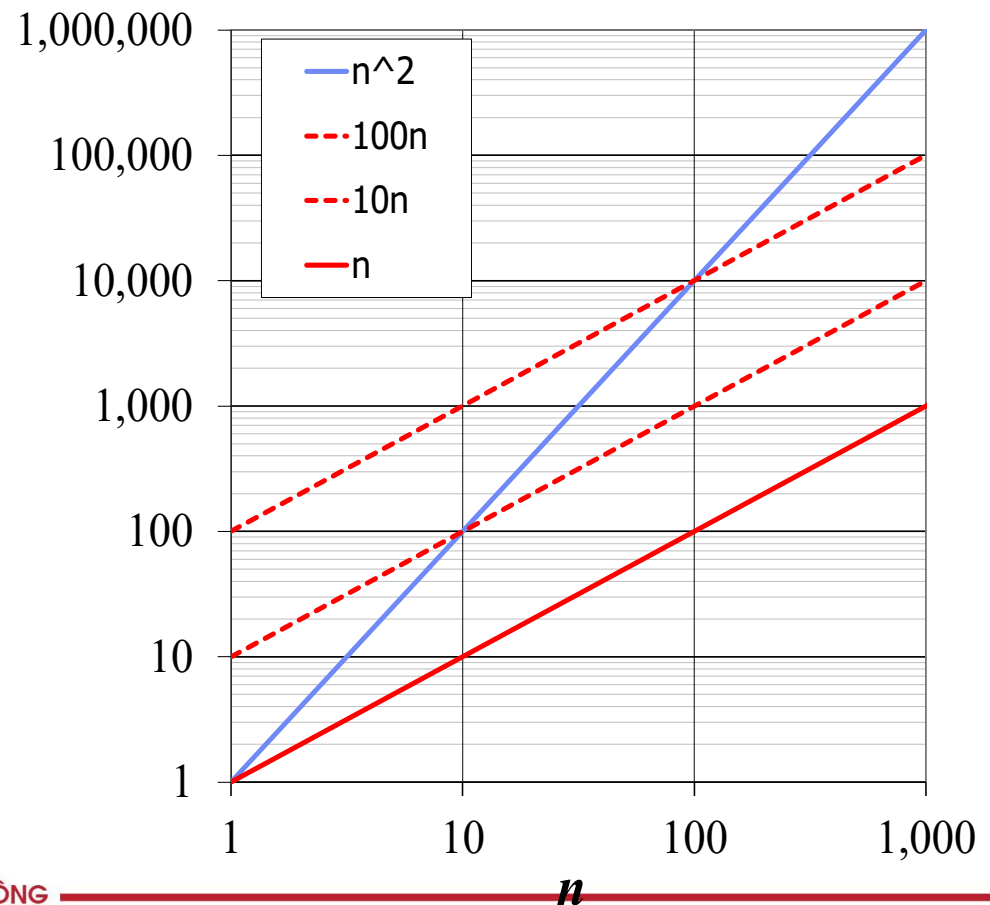
Need constants c and n_0 such that $3 \log n + 5 \leq c \log n$ for $n \geq n_0$
this is true for $c = 8$ and $n_0 = 2$

Big-Oh Examples

$O(g(n)) = \{f(n) : \exists \text{ positive constants } c \text{ and } n_0, \text{ such that } \forall n \geq n_0, \text{ we have } 0 \leq f(n) \leq cg(n)\}$

- Example 4: the function n^2 is not $O(n)$

- $n^2 \leq cn$
- $n \leq c$
- The above inequality cannot be satisfied since c must be a constant



Big-Oh and Growth Rate

- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement “ $f(n)$ is $O(g(n))$ ” means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- We can use the big-Oh notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

O Notation Examples

- All these expressions are $O(n)$:
 - $n, 3n, 61n + 5, 22n - 5, \dots$
- All these expressions are $O(n^2)$:
 - $n^2, 9n^2, 18n^2 + 4n - 53, \dots$
- All these expressions are $O(n \log n)$:
 - $n(\log n), 5n(\log 99n), 18 + (4n - 2)(\log (5n + 3)), \dots$

Ω -Omega notation

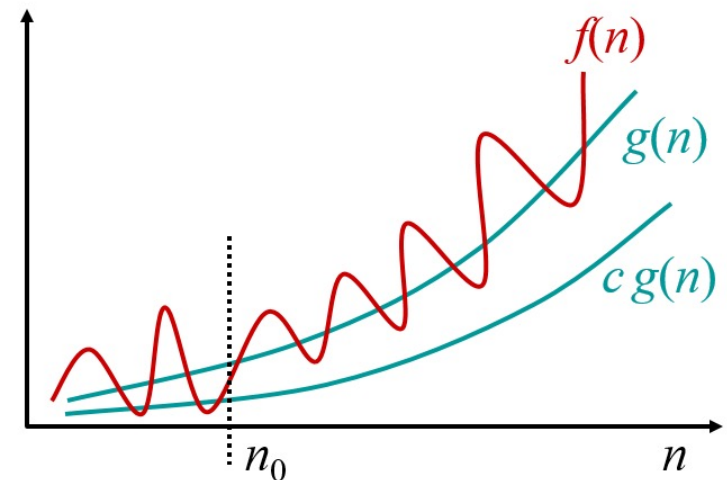
- For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$$\Omega(g(n)) = \left\{ f(n) : \begin{array}{l} \text{there exist positive constants } c \text{ and } n_0 \text{ s.t.} \\ 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

Intuitively: Set of all functions whose *rate of growth* is the same as or higher than that of $g(n)$.

- We say: $g(n)$ is asymptotic lower bound of the function $f(n)$, to within a constant factor, and write $f(n) = \Omega(g(n))$.
- $f(n) = \Omega(g(n))$ means that there exists some constant c such that $f(n)$ is always $\geq cg(n)$ for large enough n .
- $\Omega(g(n))$ is the set of functions that go to infinity no slower than $g(n)$

$$f(n) = \Theta(g(n)) \Rightarrow f(n) = \Omega(g(n))$$
$$\Theta(g(n)) \subset \Omega(g(n)).$$



Omega Examples

$$\Omega(g(n)) = \{f(n) : \exists \text{ positive constants } c \text{ and } n_0, \text{ such that} \\ \forall n \geq n_0, \text{ we have } 0 \leq cg(n) \leq f(n)\}$$

- Example 1: Show that $5n^2$ is $\Omega(n)$

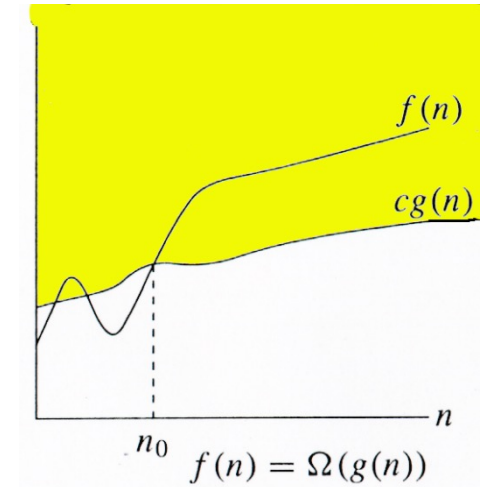
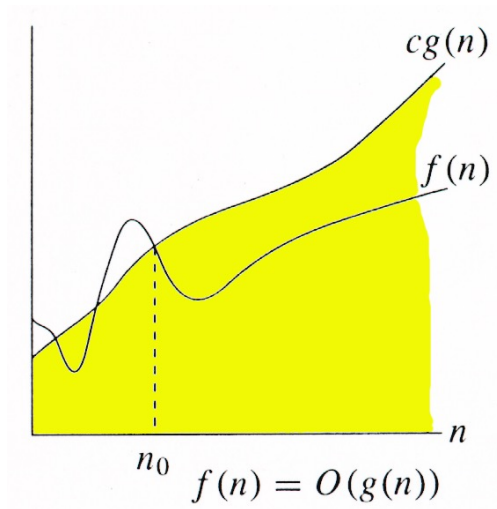
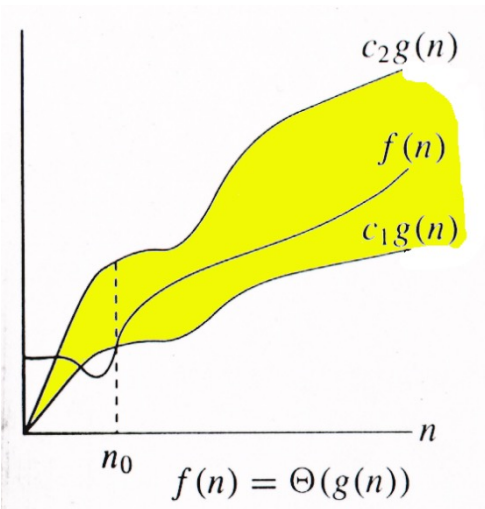
Need constants c and n_0 such that $cn \leq 5n^2$ for $n \geq n_0$
this is true for $c = 1$ and $n_0 = 1$

Comment:

- If $f(n) = \Omega(n^k)$ then $f(n) = \Omega(n^p)$ with $\forall p < k$.
- When evaluate asymptotic $f(n) = \Omega(g(n))$, we want to find function $g(n)$ with a faster growth rate as possible

- Example 2: Show that $\sqrt{n} = \Omega(\lg n)$

Asymptotic notation



Graphic examples of Θ , O , and Ω

Theorem: For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

Example: Show that $f(n) = 5n^2 = \Theta(n^2)$

Because:

- $5n^2 = O(n^2)$

$f(n)$ is $O(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \leq cg(n)$ for $n \geq n_0$

let $c = 5$ and $n_0 = 1$

- $5n^2 = \Omega(n^2)$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq cg(n)$ for $n \geq n_0$

let $c = 5$ and $n_0 = 1$

Therefore: $f(n) = \Theta(n^2)$

Algorithm analysis example

Consider sequential search algorithm to solve the problem:

- **Input:** *key*, *n* and sequence $s_1, s_2, \dots, s_n$.
- **Output:** position in the sequence that *key* is located or $n+1$ if *key* is not in the sequence.

```
int  Linear_Search(s, n, key)
{
    i=0;
    do
        i=i+1;
    while  (i<=n) or (key != si);
    return i;
}
```

Algorithm analysis example

```
int Linear_Search(s, n, key)
{
    i=0;
    do
        i=i+1;
    while (i<=n) or (key != si);
    return i;
}
```

Need to evaluate the **best, worst, average** computation time of the algorithm with an input length of n .

Apparently, the computation time of the algorithm can be evaluated by the number of executions of the statement $i = i + 1$ in the do-while loop.

Best-case running time: If $s_1 = key$, the statement $i = i + 1$ in the body of the do-while loop executes once. Therefore, the best-case running time of the algorithm is $\Theta(1)$.

Worst-case running time: If the *key* is not presented in the given sequence, the statement $i = i + 1$ executes n times. So the worst-case running time of the algorithm is $\Theta(n)$.

Algorithm analysis example

```
int Linear_Search(s, n, key)
{
    i=0;
    do
        i=i+1;
    while (i<=n) or (key != si);
    return i;
}
```

- **Average-case running time:**

- If *key* is found at position *i* of sequence ($key = s_i$) then the statement $i = i+1$ is executed *i* times ($i = 1, 2, \dots, n$),
- If *key* is not found in the sequence then the statement $i = i+1$ is executed *n* times.

→ Thus, the average number of times that the statement $i = i+1$ being executed is

$$[(1 + 2 + \dots + n) + n] / (n+1) = [n(n+1)/2 + n] / (n+1).$$

We have:

$$n/4 \leq [n(n+1)/2 + n] / (n+1) \leq n \text{ với mọi } n \geq 1.$$

Thus, the average-case running time of the algorithm is $\Theta(n)$.

CONTENTS

1. Introduction to problem
2. Algorithm and Complexity
- 3. Generating algorithm**
4. Backtracking algorithm

3. Generating algorithm

3.1. Algorithm diagram

3.2. Generate basic combinatorial configurations

- Generate binary strings of length n
- Generate m -element subsets of the set of n elements
- Generate permutations of n elements

3.1. Algorithm diagram

The generating algorithm could be applied to solve the combinatorial enumeration problem if the following two conditions are fulfilled:

- 1) An order can be specified on the set of combinations to enumerate. From there, the first and last configuration could be defined in the order specified.
- 2) Build the algorithm to give the next configuration of the current configuration (not the final one yet).

The algorithm referred to in condition 2) is called **Successive Generation Algorithm**

Generate algorithm

```
void Generate ( )  
{  
    <Build the first configuration>;  
    stop=false;  
    while (!stop)  
    {  
        <Print out the current configuration>;  
        if (the current configuration is not the final yet)  
            Successive_Generation ( );  
        else stop = true;  
    }  
}
```

<Successive_Generation> is the procedure create the next configuration of the current one in the order specified.

Note: Since the set of combinatorial configurations to enumerate is finite, we can always define an order on them. However, the order should be determined so that the successive generation algorithm could be built.

3. Generating algorithm

3.1. Algorithm diagram

3.2. Generate basic combinatorial configurations

- **Generate binary strings of length n**
- Generate m -element subsets of the set of n elements
- Generate permutations of n elements

Generate binary strings of length n

Problem: Enumerate all binary strings of length n :

$$b_1 b_2 \dots b_n, \text{ where } b_i \in \{0, 1\}.$$

Solution:

- Dictionary order:

Consider each binary string $b = b_1 b_2 \dots b_n$ as the binary representation of an integer number $p(b)$

We say binary string $b = b_1 b_2 \dots b_n$ is *previous* binary string $b' = b'_1 b'_2 \dots b'_n$ in dictionary order and denote as $b \prec b'$ if $p(b) < p(b')$.

Generate binary strings of length n

Example: When $n=3$, all binary strings of length 3 are enumerated in the dictionary order in the table as following:

b	$p(b)$
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Generate binary strings of length n

Successive_Generation algorithm:

- The first string is $0\ 0\ \dots\ 0$,
- The last string is $1\ 1\ \dots\ 1$.
- Assume $b_1\ b_2\ \dots\ b_n$ be the current string:
 - If this string consists of all 1 $\rightarrow$ finish,
 - Otherwise, the next string is obtained by adding 1 (module 2, memorize) to the current string.
- Thus, we have following rule to generate the next string:
 - Find the first i (in the order $i = n, n-1, \dots, 1$) such that $b_i = 0$.
 - Reassign $b_i = 1$ and $b_j = 0$ for all $j > i$. The newly obtained sequence will be the string to find.

Generate binary strings of length n

- Find the first i (in the order $i = n, n-1, \dots, 1$) such that $b_i = 0$.
- Reassign $b_i = 1$ and $b_j = 0$ for all $j > i$. The newly obtained sequence will be the string to find.

Example: consider binary string of length 10: $b = 1101011111$.
Find the next string:.

- We have $i = 5$. Thus, set:
 - $b_5 = 1$,
 - and $b_i = 0$, $i = 6, 7, 8, 9, 10$

→ We obtain the next string is

1101100000.

$$\begin{array}{r} 1101011111 \\ + 1 \\ \hline \end{array}$$

Successive_Generation algorithm

```
void Next_Bit_String( )
```

```
/*Generate the next binary string in the dictionary  
order
```

```
of the current string  $b_1 b_2 \dots b_n \neq 1 1 \dots 1$  */  
{
```

```
    i = n
```

```
    while (b[i] == 1):
```

```
        b[i] = 0
```

```
        i = i-1
```

```
    b[i] = 1
```

```
}
```

3. Generating algorithm

3.1. Algorithm diagram

3.2. Generate basic combinatorial configurations

- Generate binary strings of length n
- **Generate m -element subsets of the set of n elements**
- Generate permutations of n elements

Generate m -element subsets of set with n elements

Problem: Let $X = \{1, 2, \dots, n\}$. Enumerate all m -element subsets of X .

Solution:

- Lexicographic order:

Each m -element subset of X could be represented by tuples of m elements

$$a = (a_1, a_2, \dots, a_m)$$

satisfying

$$1 \leq a_1 < a_2 < \dots < a_m \leq n.$$

Generate m -element subsets of set with n elements

We say subset $a = (a_1, a_2, \dots, a_m)$ is **previous** subset $a' = (a'_1, a'_2, \dots, a'_m)$ in dictionary order and denote as $a \prec a'$, if one could find the index k ($1 \leq k \leq m$) such that:

$$a_1 = a'_1, a_2 = a'_2, \dots, a_{k-1} = a'_{k-1},$$

$$a_k < a'_k.$$

Generate m -element subsets of set with n elements

Example: Enumerate all 3-element subset of $X = \{1, 2, 3, 4, 5\}$ in dictionary order

1	2	3
1	2	4
1	2	5
1	3	4
1	3	5
1	4	5
2	3	4
2	3	5
2	4	5
3	4	5

Generate m -element subsets of set with n elements

Successive_Generation algorithm:

- The first subset is $(1, 2, \dots, m)$
- The last subset is $(n-m+1, n-m+2, \dots, n)$.
- Assume $a=(a_1, a_2, \dots, a_m)$ is the current subset but not the final yet, then its next subset in the dictionary order could be built by using the following rules:
 - Scan from the right to the left of sequence $a_1, a_2, \dots, a_m$: find the first element $a_i \neq n-m+i$
 - Replace a_i by $a_i + 1$
 - Replace a_j by $a_i + j - i$, where $j = i+1, i+2, \dots, m$

Generate m -element subsets of set with n elements

Example: $n = 6, m = 4$

Assume the current subset (1, 2, 5, 6), we need to build its next subset in the dictionary order:

- Scan from the right to the left of sequence $a_1, a_2, \dots, a_m$: find the first element $a_i \neq n-m+i$
- Replace a_i by $a_i + 1$
- Replace a_j by $a_i + j - i$, where $j = i+1, i+2, \dots, m$

- We have $i=2$:

Sequence: (1, 2, 5, 6)

Value $n-m+i$: (3, 4, 5, 6)

replace $a_2 = a_2 + 1 = 3$

$$a_3 = a_i + j - i = a_2 + 3 - 2 = 4$$

$$a_4 = a_i + j - i = a_2 + 4 - 2 = 5$$

We then obtain its next subset (1, 3, 4, 5).

Successive_Generation algorithm

```
void Next_Combination( )  
/*Generate the next subset in dictionary order of the  
subset  $(a_1, a_2, \dots, a_m) \neq (n-m+1, \dots, n)$ */  
{  
     $i = m$   
    while (a[i] == n-m+i):  
         $i = i-1$   
    a[i]= a[i] + 1  
    for j in range (i+1, m+1):  
         $a[j] = a[i] + j - i$   
}
```

3. Generating algorithm

3.1. Algorithm diagram

3.2. Generate basic combinatorial configurations

- Generate binary strings of length n
- Generate m -element subsets of the set of n elements
- **Generate permutations of n elements**

Generate permutations of n elements

Problem: Give $X = \{1, 2, \dots, n\}$, enumerate all permutations of n elements of X .

Solution:

- **Dictionary order:**

- Each permutation of n elements of X could be represented by an ordered set of n elements:

$$a = (a_1, a_2, \dots, a_n)$$

satisfy

$$a_i \in X, \quad i = 1, 2, \dots, n, \quad a_p \neq a_q, \quad p \neq q.$$

Generate permutations of n elements

We say permutation $a = (a_1, a_2, \dots, a_n)$ is previous permutation $a' = (a'_1, a'_2, \dots, a'_n)$ in dictionary order and denote as $a \prec a'$, if we could find the index k ($1 \leq k \leq n$) such that:

$$a_1 = a'_1, a_2 = a'_2, \dots, a_{k-1} = a'_{k-1},$$

$$a_k < a'_k.$$

Generate permutations of n elements

- Example: All permutation of 3 elements of $X = \{1, 2, 3\}$ could be enumerated in dictionary order as following:

1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

Generate permutations of n elements

Successive_Generation algorithm:

- The first permutation: $(1, 2, \dots, n)$
- The last permutation: $(n, n-1, \dots, 1)$.
- Assume $a = (a_1, a_2, \dots, a_n)$ is not the last permutation, then its next permutation could be built using the following rules:
 - Find from the right to left: first index j satisfying $a_j < a_{j+1}$ (in other word: j is the max index satisfying $a_j < a_{j+1}$);
 - Find a_k be the smallest number among those on the right of a_j in the sequence and *greater than* a_j ;
 - Swap a_j and a_k ;
 - Invert the sequence from a_{j+1} to a_n .

Generate permutations of n elements

Example: Assume the current permutation $(3, 6, 2, 5, 4, 1)$, need to create its next permutation in the dictionary order.

- Find from the right to left: first index j satisfying $a_j < a_{j+1}$ (in other word: j is the max index satisfying $a_j < a_{j+1}$);
 - Find a_k be the smallest number among those on the right of a_j in the sequence and *greater than* a_j ;
 - Swap a_j and a_k ;
 - Invert the sequence from a_{j+1} to a_n .
- We have index $j = 3$ ($a_3 = 2 < a_4 = 5$).
 - The smallest number among those on the right of a_3 and greater than a_3 is $a_5 = 4$. Swap a_3 and a_5 in the current permutation $(3, 6, \mathbf{2}, 5, \mathbf{4}, 1)$
we obtain $(3, 6, \mathbf{4}, 5, \mathbf{2}, 1)$,
 - Finally, invert the sequence $a_4 a_5 a_6$, we obtain the next permutation is $(3, 6, 4, \mathbf{1}, \mathbf{2}, \mathbf{5})$.

Successive_Generation algorithm

```
void Next_Permutation ( )
{
    /*Generate next permutation  $(a_1, a_2, \dots, a_n) \neq (n, n-1, \dots, 1)$  */
    // Find j being max index satisfying  $a_j < a_{j+1}$  :
    j=n-1; while (a_j > a_{j+1}) j=j-1;
    // Find a_k being smallest number among those on the right a_j and greater
    //than a_j :
    k=n; while (a_j > a_k) k=k-1;
    Swap(a_j, a_k); //swap a_j and a_k
    // Invert the sequence a_{j+1} to a_n :
    r=n; s=j+1;
    while ( r>s )
    {
        Swap(a_r, a_s); // swap a_r and a_s
        r=r-1; s= s+1;
    }
}
```


CONTENTS

1. Introduction to problem
2. Algorithm and Complexity
3. Generating algorithm
- 4. Backtracking algorithm**

Backtracking

4.1. Algorithm diagram

4.2. Generate basic combinatorial configurations

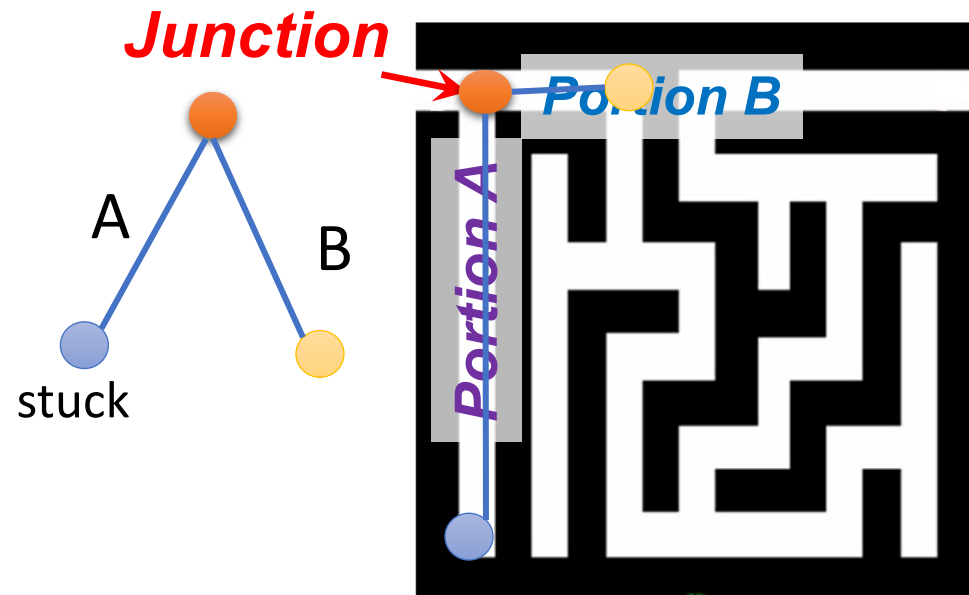
- Generate binary strings of length n
- Generate m -element subsets of the set of n elements
- Generate permutations of n elements

Backtracking idea

- A clever form of exhaustive search.
- Backtracking is a technique used to solve problems with a large search space, by systematically trying and eliminating possibilities.

Example of backtracking would be going through a maze.

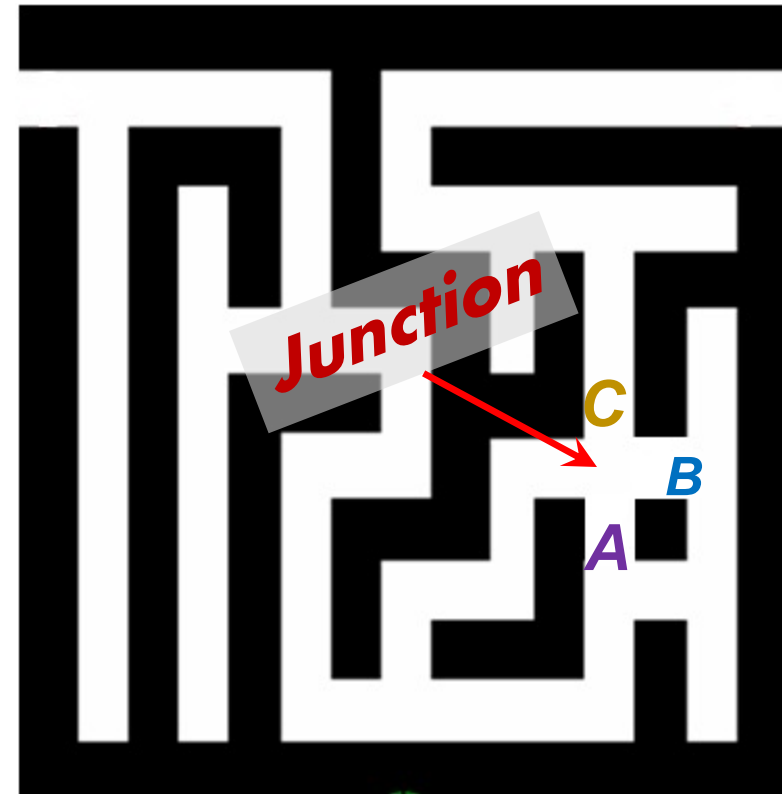
- At some point in a maze, you might have two options of which direction to go:
 - One strategy would be to try going through **Portion A** of the maze.
 - If you get stuck before you find your way out, then you "*backtrack*" to the junction.
 - At this point in time you know that **Portion A** will **NOT** lead you out of the maze,



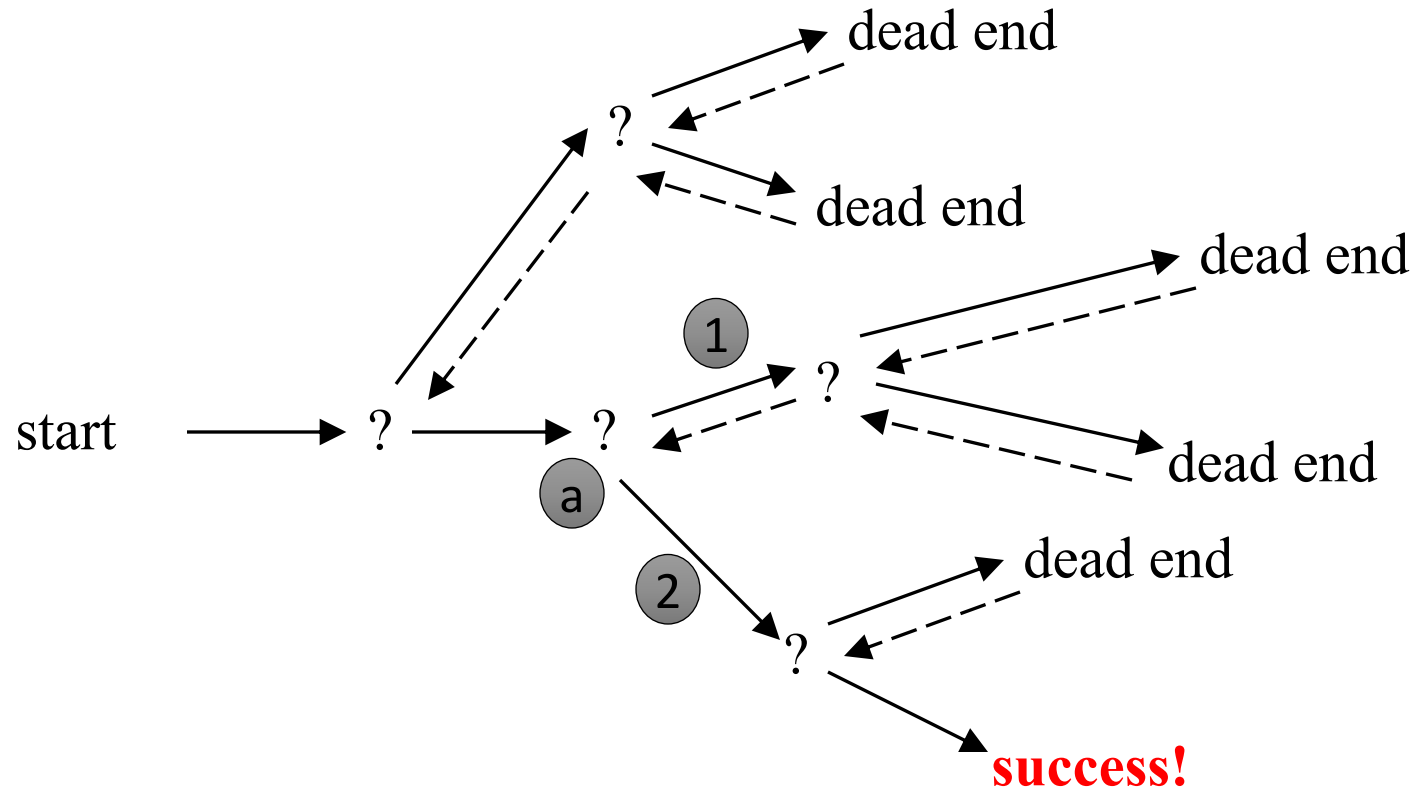
so you then start searching in

Backtracking idea

- Clearly, at a single junction you could have even more than 2 choices.
- The backtracking strategy says to try each choice, one after the other,
 - if you ever get stuck, "*backtrack*" to the junction and try the next choice.
- If you try all choices and never found a way out, then there IS no solution to the maze.



Backtracking (animation)



Backtracking idea

- Dealing with the maze:
 - From your start point, you will iterate through each possible starting move.
 - From there, you recursively move forward.
 - If you ever get stuck, the recursion takes you back to where you were, and you try the next possible move.
- Make sure you don't try too many possibilities,
 - Mark which locations in the maze have been visited already so that no location in the maze gets visited twice.
 - If a place has already been visited, there is no point in trying to reach the end of the maze from there again.

Backtracking diagram

- **Enumeration problem (Q):** Given $A_1, A_2, \dots, A_n$ be finite sets. Denote $A = A_1 \times A_2 \times \dots \times A_n = \{ (a_1, a_2, \dots, a_n) : a_i \in A_i, i=1, 2, \dots, n \}$.
Assume P is a property on the set A . The problem is to enumerate all elements of the set A that satisfies the property P :
$$D = \{ a = (a_1, a_2, \dots, a_n) \in A : a \text{ satisfies property } P \}.$$
- Elements of the set D are called **feasible solution** (lời giải chấp nhận được).

Backtracking diagram

All basic combinatorial enumeration problem could be rephrased in the form of Enumeration problem (Q).

Example:

- The problem of enumerating all binary string of length n leads to the enumeration of elements of the set:

$$B^n = \{(a_1, \dots, a_n): a_i \in \{0, 1\}, i=1, 2, \dots, n\}.$$

- The problem of enumerating all m -element subsets of set $N = \{1, 2, \dots, n\}$ requires to enumerate elements of the set:

$$S(m, n) = \{(a_1, \dots, a_m) \in N^m: 1 \leq a_1 < \dots < a_m \leq n\}.$$

- The problem of enumerating all permutations of natural numbers $1, 2, \dots, n$ requires to enumerate elements of the set

$$\Pi_n = \{(a_1, \dots, a_n) \in N^n: a_i \neq a_j; i \neq j\}.$$

Partial solution

The solution to the problem is an ordered tuple of n elements $(a_1, a_2, \dots, a_n)$, where $a_i \in A_i$, $i = 1, 2, \dots, n$.

Definition. The k -level partial solution ($0 \leq k \leq n$) is an ordered tuple of k elements

$$(a_1, a_2, \dots, a_k),$$

where $a_i \in A_i$, $i = 1, 2, \dots, k$.

- When $k = 0$, 0-level partial solution is denoted as $()$, and called as the empty solution.
- When $k = n$, we have a complete solution to a problem.

Backtracking diagram

Backtracking algorithm is built based on the construction each component of solution one by one.

- Algorithm starts with empty solution ().
- Based on the property P , we determine which elements of set A_1 could be selected as the first component of solution. Such elements are called as **candidates** for the first component of solution. Denote candidates for the first component of solution as S_1 . Take an element $a_1 \in S_1$, insert it into empty solution, we obtain 1-level partial solution: (a_1) .

Backtracking diagram

- General step: Assume we have $k-1$ level partial solution:

$$(a_1, a_2, \dots, a_{k-1}),$$

Now we need to build k -level partial solution:

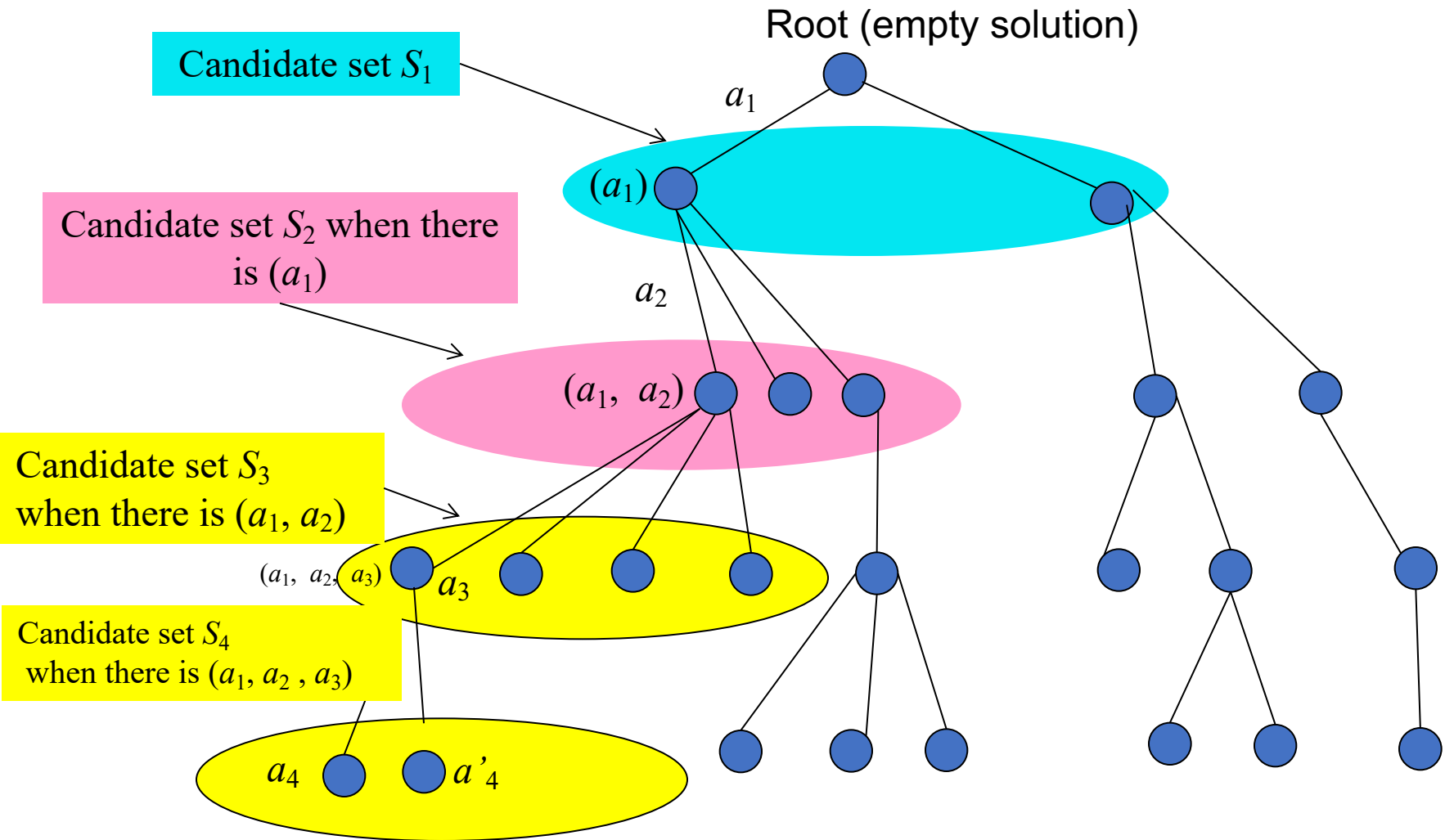
$$(a_1, a_2, \dots, a_{k-1}, \mathbf{a_k})$$

- Based on the property P, we determine which elements of set A_k could be selected as the k^{th} component of solution.
- Such elements are called as candidates for the k^{th} position of solution when $k-1$ first components have been chosen as $(a_1, a_2, \dots, a_{k-1})$. Denote these candidates by S_k .
- Consider 2 cases:
 - $S_k \neq \emptyset$
 - $S_k = \emptyset$

Backtracking diagram

- $S_k \neq \emptyset$: Take $a_k \in S_k$ to insert it into current $(k-1)$ -level partial solution $(a_1, a_2, \dots, a_{k-1})$, we obtain k -level partial solution $(a_1, a_2, \dots, a_{k-1}, a_k)$. Then
 - If $k = n$, then we obtain a complete solution to the problem,
 - If $k < n$, we continue to build the $(k+1)^{\text{th}}$ component of solution.
- $S_k = \emptyset$: It means the partial solution $(a_1, a_2, \dots, a_{k-1})$ can not continue to develop into the complete solution. In this case, we **backtrack** to find new candidate for $(k-1)^{\text{th}}$ position of solution (note: this new candidate must be an element of S_{k-1})
 - If one could find such candidate, we insert it into $(k-1)^{\text{th}}$ position, then continue to build the k^{th} component.
 - If such candidate could not be found, we **backtrack** one more step to find new candidate for $(k-2)^{\text{th}}$ position, ... If backtrack till the empty solution, we still can not find new candidate for 1^{st} position, then the algorithm is finished.

Decision tree for backtracking



Backtracking algorithm (recursive)

```
void Try(int k)
{
    <Build  $S_k$  as the set to consist of candidates
    for the  $k^{\text{th}}$  component of solution>;
    for  $y \in S_k$  //Each candidate  $y$  of  $S_k$ 
    {
         $a_k = y$ ;
        if  $(k == n)$  then <Record  $(a_1, a_2, \dots, a_k)$ 
        as a complete solution >;
        else Try( $k+1$ );
        Return the variables to their old states;
    }
}
```

The call to execute backtracking algorithm: **Try(1);**

- If only one solution need to be found, then it is necessary to find a way to terminate the nested recursive calls generated by the call to Try(1) once the first solution has just been recorded.
- If at the end of the algorithm, we obtain no solution, it means that the problem does not have any solution.

Backtracking algorithm (not recursive)

```
void Backtraking ( )
```

```
{
     $k=1$ ;
    <Build  $S_k$ >;
    while  $(k > 0)$  {
        while  $(S_k \neq \emptyset)$  {
             $a_k \leftarrow S_k$ ; // Take  $a_k$  from  $S_k$ 
            if  $<(k == n) >$  then <Record  $(a_1, a_2, \dots, a_k)$  as a
            complete solution >;

            else {
                 $k = k+1$ ;
                <Build  $S_k$ >;
            }
        }
         $k = k - 1$ ; // Backtracking
    }
}
```

The call to execute backtracking algorithm:
Backtraking ();

Two key issues

- In order to implement a backtracking algorithm to solve a specific combinatorial problems, we need to solve the following two basic problems:
 - Find algorithms to build candidate sets S_k
 - Find a way to describe these sets so that you can implement the operation to **enumerate all their elements** (implement the loop **for** $y \in S_k$).
 - The **efficiency of the enumeration algorithm** depends on whether we can accurately identify these candidate sets.

Note

- If the length of complete solution is not known in advanced, and solutions are not needed to have the same length:

```
void Try(int k)
{
    <Build  $S_k$  as the set to consist of candidates for the  $k^{\text{th}}$  component of solution>;
    for  $y \in S_k$  //Each candidate  $y$  of  $S_k$ 
    {
         $a_k = y$ ;
        if ( $k == n$ ) then <Record ( $a_1, a_2, \dots, a_k$ ) as a complete solution >;
        else Try( $k+1$ );
        Return the variables to their old states;
    }
}
```

- Then we need to modify statement

```
if ( $k == n$ ) then <Record ( $a_1, a_2, \dots, a_k$ ) as a complete solution >;
else Try( $k+1$ );
```

to

```
if <( $a_1, a_2, \dots, a_k$ ) is a complete solution> then <Record ( $a_1, a_2, \dots, a_k$ ) as a
complete solution >;
else Try( $k+1$ );
```

→ Need to build a function to check whether $(a_1, a_2, \dots, a_k)$ is the complete solution.

4. Backtracking

4.1. Algorithm diagram

4.2. Generate basic combinatorial configurations

- **Generate binary strings of length n**
- Generate m -element subsets of the set of n elements
- Generate permutations of n elements

Example 1: Enumerate all binary string of length n

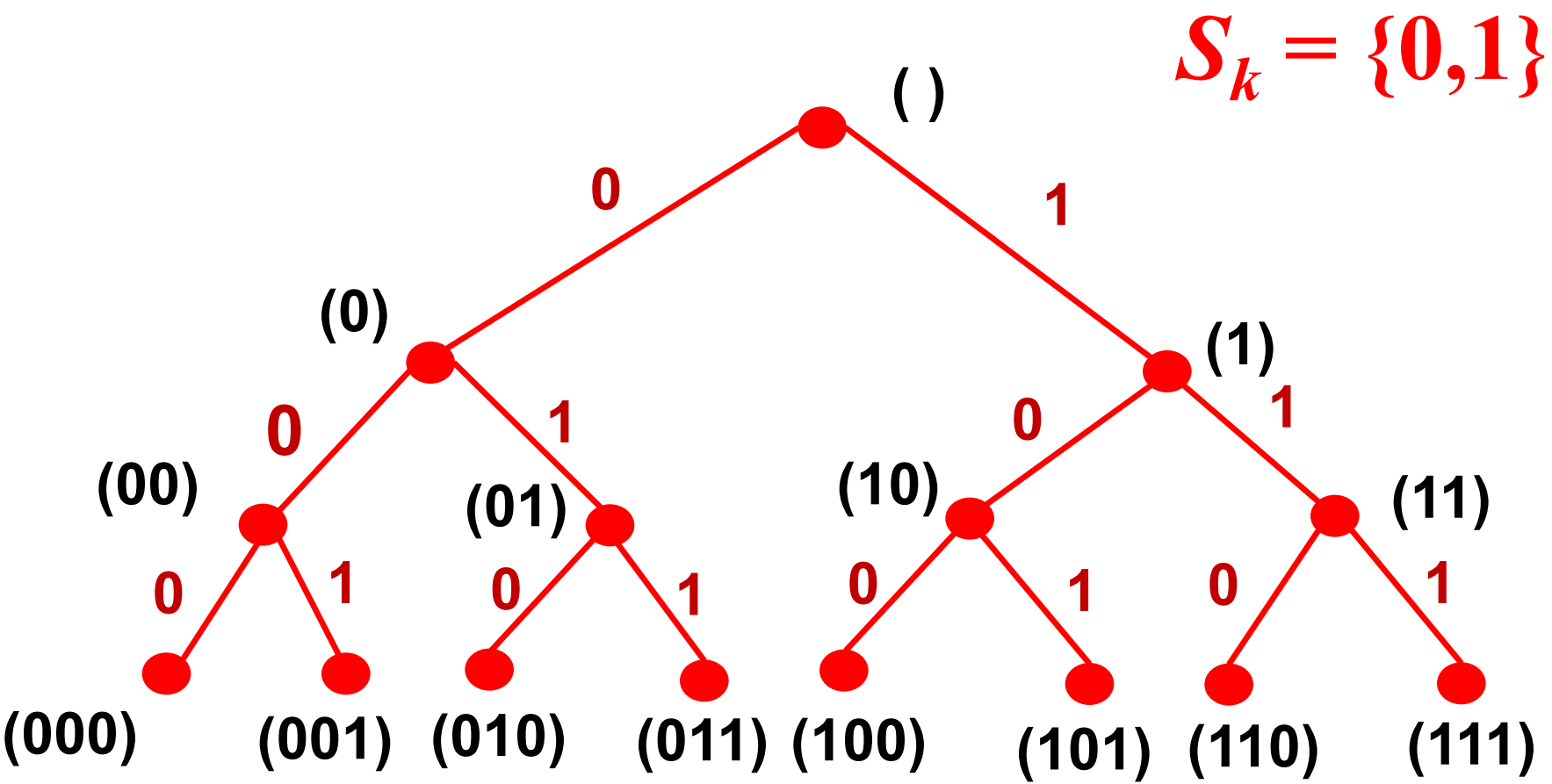
- Problem to enumerate all binary string of length n leads to the enumeration of all elements of the set:

$$A^n = \{(a_1, \dots, a_n): a_i \in \{0, 1\}, i=1, 2, \dots, n\}.$$

- We consider how to solve two issue keys to implement backtracking algorithm:
 - **Build candidate set S_k :** We have $S_1 = \{0, 1\}$. Assume we have binary string of length $k-1$ $(a_1, \dots, a_{k-1})$, then $S_k = \{0, 1\}$. Thus, the candidate sets for each position of the solution are determined.
 - **Implement the loop to enumerate all elements of S_k :** we can use the loop `for`

`for (y=0; y<=1; y++) in C/C++`

Decision tree to enumerate binary strings of length 3



Program in C++ (Recursive)

```
#include <iostream>
using namespace std;
int n, count;
int a[100];

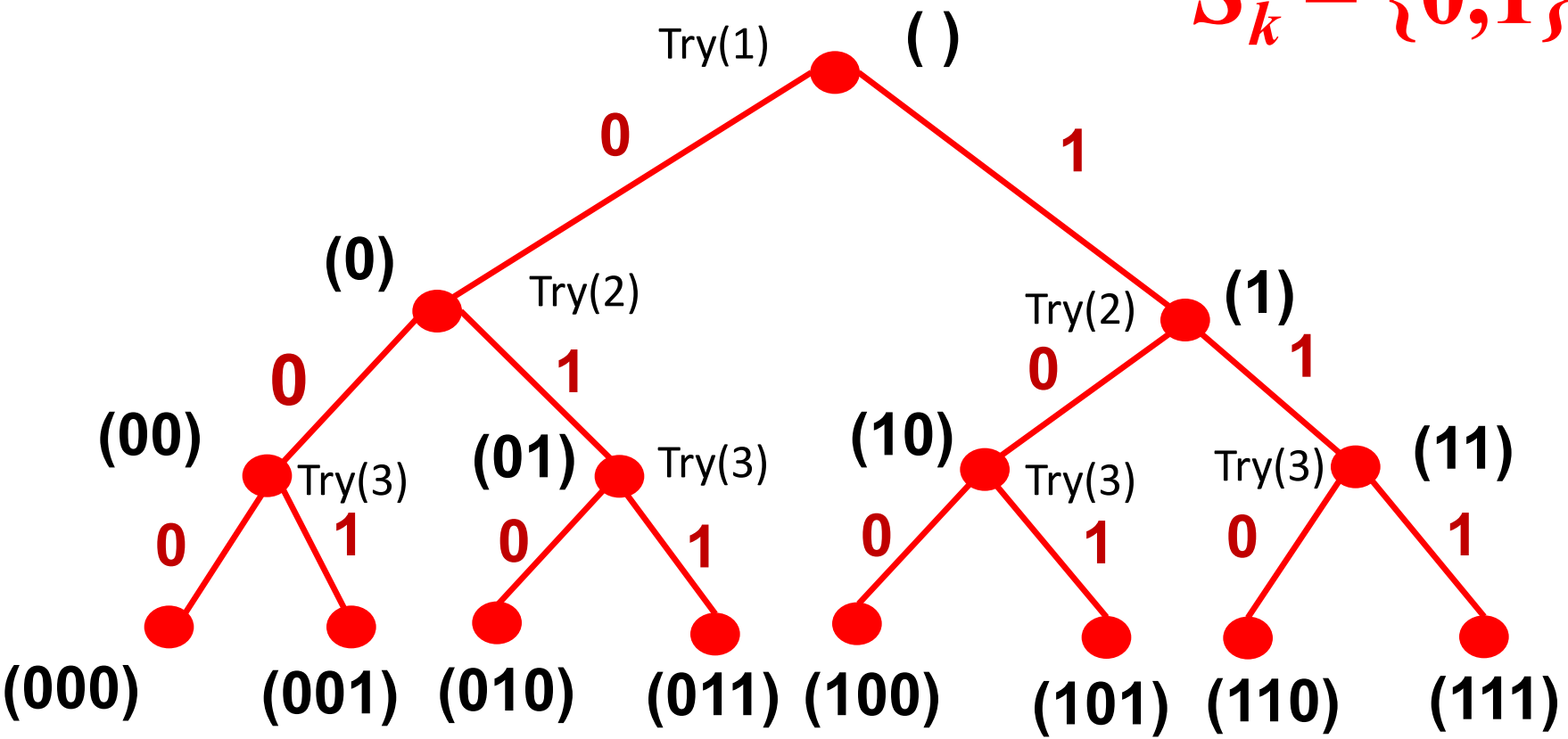
void PrintSolution()
{
    int i, j;
    count++;
    cout<<"String # " <<count<<": ";
    for (i=1 ; i<= n ;i++)
    {
        j=a[i];
        cout<<j<<"    ";
    }
    cout<<endl;
}
```

```
void Try(int k)
{
    for (int j = 0; j<=1; j++)
    {
        a[k] = j;
        if (k == n) PrintSolution();
        else Try(k+1);
    }
}

int main()
{
    cout<<"Enter n = ";cin>>n;
    count = 0; Try(1);
    cout<<"Number of strings "<<count;
}
```

Decision tree to enumerate binary strings of length 3

$S_k = \{0,1\}$



Program in C++ (non recursive)

```
#include <iostream>
using namespace std;
int n, count, k;
int a[100], s[100];

void PrintSolution()
{
    int i, j;
    count++;
    cout<<"String # " << count<<": ";
    for (i=1 ; i<= n ;i++)
        cout<<a[i]<<" ";

    cout<<endl;
}
```

```
void GenerateString()
{
    k=1; s[k]=0;
    while (k > 0)
    {
        while (s[k] <= 1)
        {
            a[k]=s[k];
            s[k]=s[k]+1;
            if (k==n) PrintSolution();
            else
            {
                k++; s[k]=0;
            }
        }
        k--; // BackTrack
    }
}
```

Program in C++ (non recursive)

```
int main() {  
    cout<<"Enter value of  n = ";cin>>n;  
    count = 0; GenerateString();  
    cout<<"Number of strings = "<<count<<endl;  
}
```

4. Backtracking

4.1. Algorithm diagram

4.2. Generate basic combinatorial configurations

- Generate binary strings of length n
- Generate m -element subsets of the set of n elements
- Generate permutations of n elements

Example 2. Generate m -element subsets of the set of n elements

Problem: Enumerate all m -element subsets of the set n elements $N = \{1, 2, \dots, n\}$.

Example: Enumerate all 3-element subsets of the set 5 elements $N = \{1, 2, 3, 4, 5\}$

Solution: $(1, 2, 3), (1, 2, 4), (1, 2, 5), (1, 3, 4), (1, 3, 5), (1, 4, 5), (2, 3, 4), (2, 3, 5), (2, 4, 5), (3, 4, 5)$

➔ Equivalent problem: Enumerate all elements of set:

$$S(m, n) = \{(a_1, \dots, a_m) \in N^m : 1 \leq a_1 < \dots < a_m \leq n\}$$

Example 2. Generate m -element subsets of the set of n elements

We consider how to solve two issue keys to implement backtracking:

- **Build candidate set S_k :**

- With the condition: $1 \leq a_1 < a_2 < \dots < a_m \leq n$

we have $S_1 = \{1, 2, \dots, n-(m-1)\}$.

- Assume the current subset is $(a_1, \dots, a_{k-1})$, with the condition $a_{k-1} < a_k < \dots < a_m \leq n$, we have $S_k = \{a_{k-1}+1, a_{k-1}+2, \dots, n-(m-k)\}$.

- **Implement the loop to enumerate all elements of S_k :** we can use the loop `for`

`for (j=a[k-1]+1; j<=n-m+k; j++) ..`

Program in C++ (Recursive)

```
#include <iostream>
using namespace std;

int n, m, count;
int a[100];

void PrintSolution() {
    int i;
    count++;
    cout<<"The subset #" <<count<<": ";
    for (i=1 ; i<= m ;i++)
        cout<<a[i]<<" ";
    cout<<endl;
```

```
void Try(int k){
    int j;
    for (j = a[k-1] +1; j<= n-m+k; j++) {
        a[k] = j;
        if (k==m) PrintSolution();
        else Try(k+1);
    }
}

int main() {
    cout<<"Enter n, m = "; cin>>n; cin>>m;
    a[0]=0; count = 0; Try(1);
    cout<<"Number of " <<m<<"-element
subsets of set " <<n<<" elements =
" <<count<<endl;
}
```

Program in C++ (Non Recursive)

```
#include <iostream>
using namespace std;
```

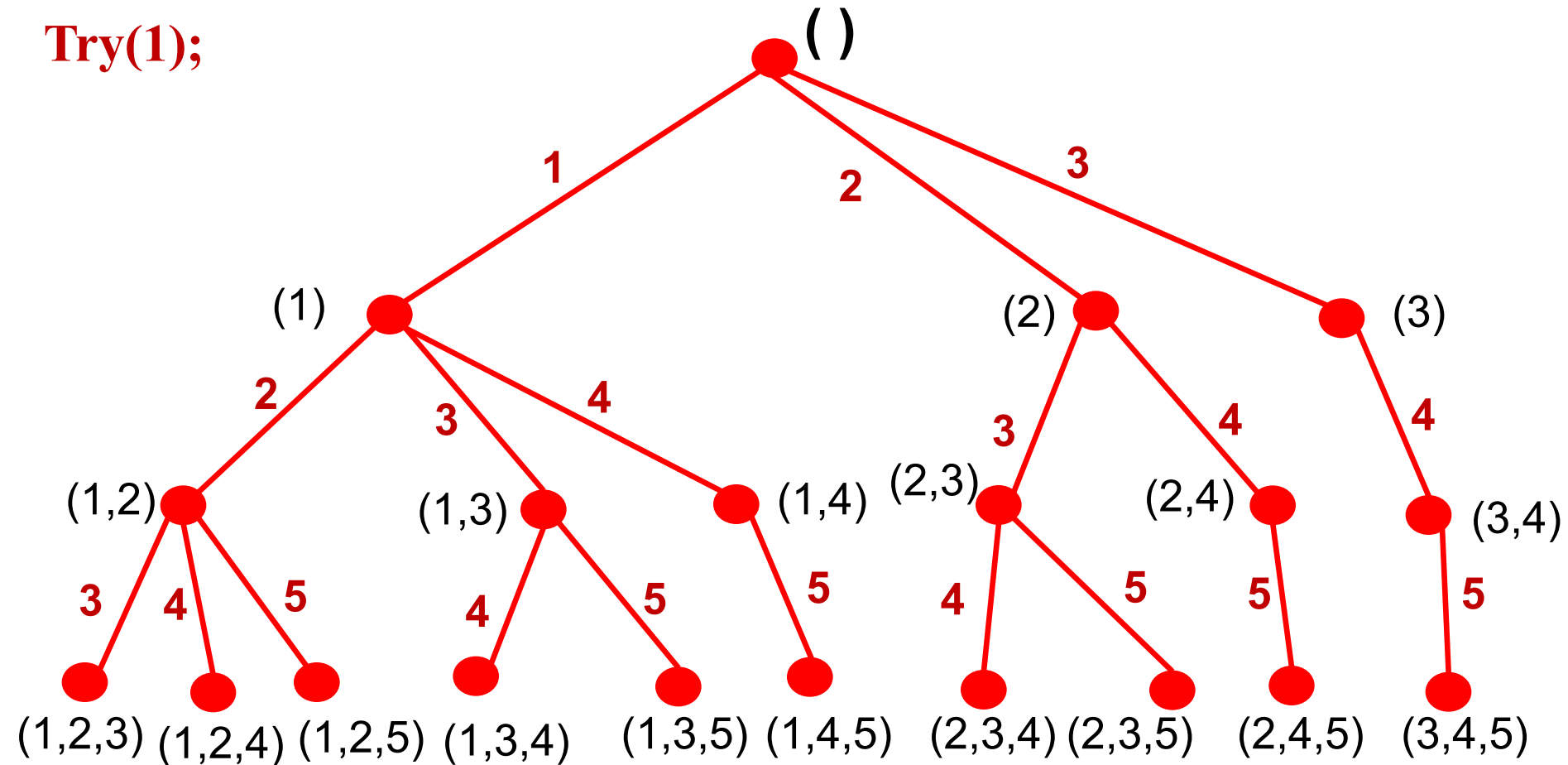
```
int n, m, count, k;
int a[100], s[100];
void PrintSolution() {
    int i;
    count++;
    cout<<"The subset #" <<count<<": ";
    for (i=1 ; i<= m ;i++)
        cout<<a[i]<<" ";
    cout<<endl;
}
```

```
void MSet()
{
    k=1; s[k]=1;
    while(k>0){
        while (s[k]<= n-m+k) {
            a[k]=s[k]; s[k]=s[k]+1;
            if (k==m) PrintSolution();
            else { k++; s[k]=a[k-1]+1; }
        }
        k--;
    }
}

int main() {
    cout<<"Enter n, m = "; cin>>n; cin>>m;
    a[0]=0; count = 0; MSet();
    cout<<"Number of "<<m<<"-element
    subsets of set "<<n<<" elements =
    "<<count<<endl;
```

Decision tree $S(5,3)$

Try(1);



$$S_k = \{a_{k-1}+1, a_{k-1}+2, \dots, n-(m-k)\}$$

4. Backtracking

4.1. Algorithm diagram

4.2. Generate basic combinatorial configurations

- Generate binary strings of length n
- Generate m -element subsets of the set of n elements
- **Generate permutations of n elements**

Example 3. Enumerate permutations

Permutation set of natural numbers $1, 2, \dots, n$ is the set:

$$\Pi_n = \{(x_1, \dots, x_n) \in N^n: x_i \neq x_j, i \neq j\}.$$

Problem: Enumerate all elements of Π_n

Example 3. Enumerate permutations

- Build candidate set S_k :
 - Actually $S_1 = N$. Assume we have current partial permutation $(a_1, a_2, \dots, a_{k-1})$, with the condition $a_i \neq a_j$, for all $i \neq j$, we have
$$S_k = N \setminus \{ a_1, a_2, \dots, a_{k-1} \}.$$

Describe S_k

Build function to detect candidates:

```
bool candidate(int j, int k)
{
    //function returns true if and only if  $j \in S_k$ 
    int i;
    for (i=1; i++; i<=k-1)
        if (j == a[i]) return false;
    return true;
}
```

Example 3. Enumerate permutations

- Implement the loop to enumerate all elements of S_k :

```
for (j=1; j <= n; j++;)
    if (candidate(j, k))
    {
        // j is candidate for position  $k^{th}$ 
        . . .
    }
```

Program in C++ (Recursive)

```
#include <iostream>
using namespace std;
```

```
int n, m, count;
int a[100];
```

```
int PrintSolution() {
    int i, j;
    count++;
    cout<<"Permutation #"<<count<<": ";
    for (i=1 ; i<= n ;i++)
        cout<<a[i]<<" ";
    cout<<endl;
}
```

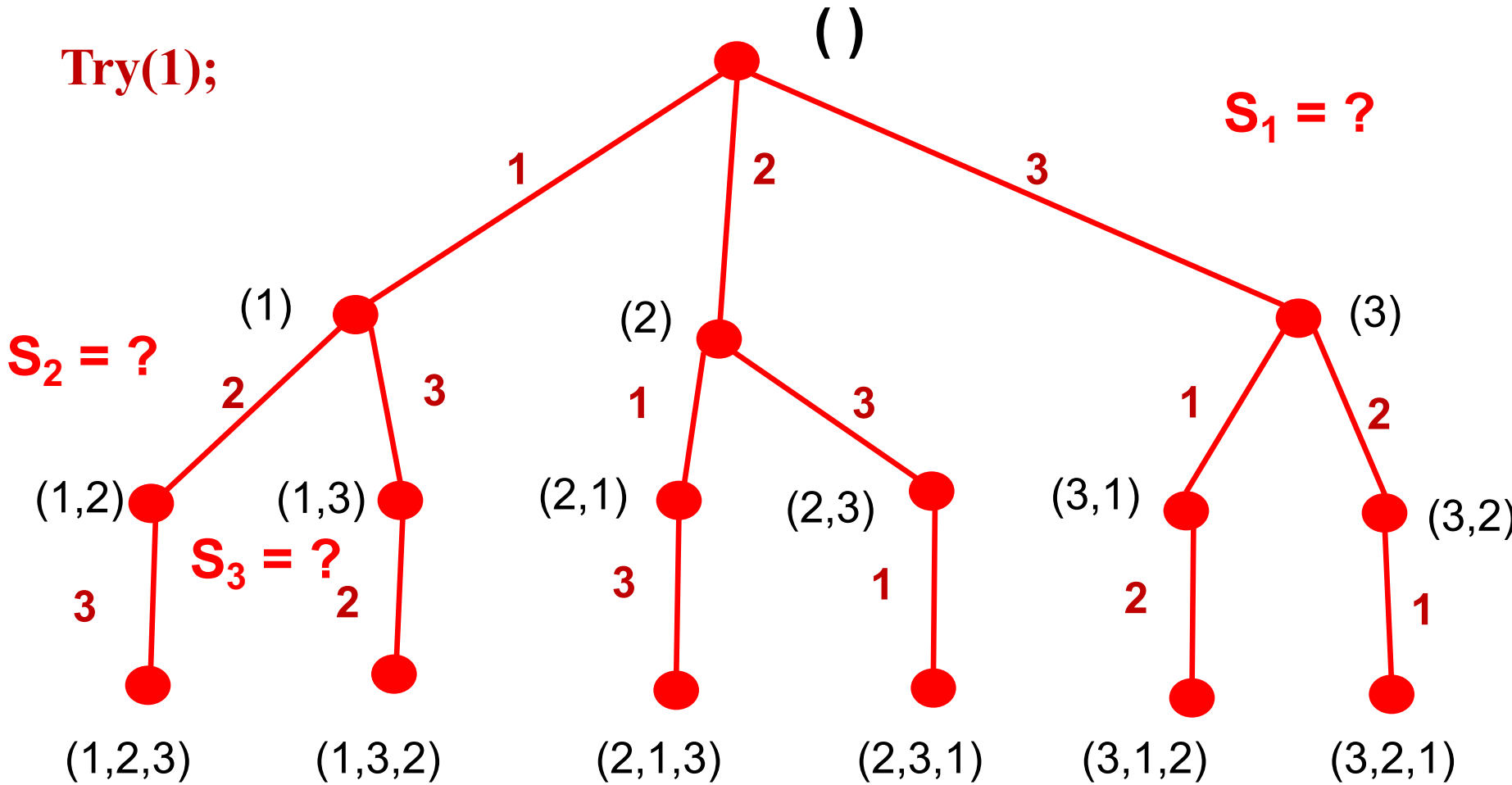
```
bool candidate(int j, int k)
{
    int i;
    for (i=1; i<=k-1; i++)
        if (j == a[i])
            return false;
    return true;
}
```

Program in C++ (Recursive)

```
void Try(int k)
{
    int j;
    for (j = 1; j<=n; j++)
        if (candidate(j,k))
        {
            a[k] = j;
            if (k==n) PrintSolution( );
            else Try(k+1);
        }
}

int main() {
    cout<<("Enter n = "); cin>>n;
    count = 0; Try(1);
    cout<<"Number of permutations = " << count;
}
```

Decision tree to enumerate permutations of 1, 2, 3



$$S_k = N \setminus \{ a_1, a_2, \dots, a_{k-1} \}$$